

GUIA DE ESTUDO — PROBABILIDADE E PYTHON

Consulta rápida para prova, baseada no material enviado: funções **random**, **itertools**, permutações, amostragem com pesos e distribuição normal.

1. MAPA RÁPIDO — QUAL FUNÇÃO USAR?

Função	Use quando...	Atenção
<code>random.uniform(a,b)</code>	número real uniforme	intervalo
<code>random.choice(lista)</code>	1 item aleatório	uma escolha
<code>random.choices(...)</code>	vários itens	pode repetir; usa weights
<code>random.sample(...)</code>	vários itens	sem repetição
<code>random.gauss(μ,σ)</code>	distribuição normal	novo sorteio no laço
<code>product(...)</code>	todas combinações	use * com <code>.values()</code>
<code>permutations(...)</code>	ordem importa	ex.: pódio
<code>math.factorial(n)</code>	n elementos em ordem	n!
<code>math.perm(n,r)</code>	r posições ordenadas	P(n,r)
<code>range(início,fim)</code>	sequência	fim não entra

2. REGRAS QUE MAIS CAEM

- **range**: o limite final não entra. Para 1 a 60: `range(1, 61)`.
- **Par**: `numero % 2 == 0`. Divisibilidade: `numero % divisor == 0`.
- **Ordem importa?** Sim → permutação. Não → combinação.
- **Repetição?** `choices` pode repetir; `sample` não repete.
- **Desempacotamento**: * transforma uma tupla/lista em argumentos separados.
- **Função**: `funcao(valor)`. Acesso a item: `lista[0]`.

3. UNIFORM — random.uniform

Distribuição uniforme: qualquer valor no intervalo tem a mesma chance de ser escolhido.

```
import random
espaco_amostral = (10, 45)
resultado = random.uniform(*espaco_amostral)
print(f"{resultado:.2f} minutos")
```

■ **DICA/ERRO**: O * desempacota a tupla: `random.uniform(10, 45)`.

4. DADO — random.choice + %

Dado: $S = \{1,2,3,4,5,6\}$. Para par, resto da divisão por 2 deve ser zero.

```
import random
espaco_amostral = range(1, 7)
resultado = random.choice(espaco_amostral)
```

```
if resultado % 2 == 0:
    print(f"{resultado} é par")
else:
    print(f"{resultado} é ímpar")
```

■ **DICA/ERRO**: `range(1, 6)` exclui o 6. Use `range(1, 7)`.

5. TRÊS MOEDAS — itertools.product

Cada moeda tem 2 resultados. Três lançamentos: $2^3 = 8$ sequências.

```
from itertools import product
espaco_amostral = list(
    product(["cara", "coroa"], repeat=3)
)
resultado = random.choice(espaco_amostral)
```

■ **DICA/ERRO**: `repeat=3` para três lançamentos. `list(...)` permite usar `len`.

6. COMBINAÇÕES DE CATEGORIAS — product

4 pães × 5 recheios × 3 bebidas = **60** combinações.

```
lanche = {  
    "paes": [...],  
    "recheios": [...],  
    "bebidas": [...]  
}  
espaco_amostral = list(product(*lanche.values()))  
len(espaco_amostral) # 60
```

■ **DICA/ERRO:** `keys()` = chaves; `values()` = opções. Não use `len(lanche)`: isso daria 3 categorias.

7. LIVROS — `math.factorial`

5 livros diferentes em uma fila: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

```
import math
numero_livros = 5
numero_maneiras = math.factorial(numero_livros)
```

■ **DICA/ERRO:** Função usa parênteses: `math.factorial(5)`, não `math.factorial[5]`.

8. PÓDIO — `math.perm` + `permutations`

7 corredores e 3 posições. A ordem importa: $P(7,3) = 7!/(7-3)! = 7 \times 6 \times 5 = 210$.

```
import math
from itertools import permutations

corredores = ["Ana", "Bruno", "Carla", "Diego", "Elisa", "Fabio", "Gabi"]
numero_podios = math.perm(len(corredores), 3)
podios = list(permutations(corredores, 3))
```

■ **DICA/ERRO:** Se a ordem muda o resultado, é permutação. Ex.: Ana-Bruno-Carla $\neq$ Bruno-Ana-Carla.

9. PESOS — `random.choices`

95% boas e 5% defeituosas. Para 1000 sorteios, use pesos [95,5].

```
espaco_amostral = ["boa", "defeituosa"]
pesos = [95, 5]
amostra = random.choices(
    espaco_amostral, weights=pesos, k=1000
)
defeituosas = amostra.count("defeituosa")
porcentagem = defeituosas / 1000 * 100
```

■ **DICA/ERRO:** `weights` indica os pesos; `k` indica quantos sorteios. `choices` pode repetir.

10. MEGA-SENA — `random.sample`

6 números diferentes entre 1 e 60. Sem repetição $\rightarrow$ `sample`.

```
import random
espaco_amostral = range(1, 61)
resultado = random.sample(espaco_amostral, k=6)
resultado.sort()
```

■ **DICA/ERRO:** Use `range(1, 61)` para incluir 60. `choices` permitiria repetição.

11. DISTRIBUIÇÃO NORMAL — `random.gauss`

Alturas: média = 1,70 m; desvio padrão = 0,08. $X \sim N(1,70; 0,08)$.

```
import random
media = 1.70
desvio_padrao = 0.08

for numero_aluno in range(1, 6):
    altura = random.gauss(media, desvio_padrao)
    print(f"Aluno {numero_aluno}: {altura:.2f} m")
```

■ **DICA/ERRO:** Não existe `random.normal`. Use `random.gauss` ou `random.normalvariate`. O sorteio fica dentro do `for`.

12. PIZZA — `product` + `choice`

3 tamanhos $\times$ 4 sabores $\times$ 2 bordas = **24** pizzas.

```
pizza = {
    "tamanhos": ["P", "M", "G"],
    "sabores": ["calabresa", "mussarela", "frango", "portuguesa"],
    "bordas": ["tradicional", "recheada"]
}
espaco_amostral = list(product(*pizza.values()))
len(espaco_amostral) # 24
resultado = random.choice(espaco_amostral)
```

13. DECISÕES RÁPIDAS NA PROVA

Pergunta	Resposta
Quero 1 item aleatório	<code>random.choice</code>
Quero vários e pode repetir	<code>random.choices</code>
Quero vários sem repetir	<code>random.sample</code>
Quero número real uniforme	<code>random.uniform</code>
Quero distribuição normal	<code>random.gauss</code>
Quero todas as combinações	<code>itertools.product</code>
A ordem importa	<code>itertools.permutations</code>
Todos os n elementos em ordem	<code>math.factorial(n)</code>
r posições entre n, com ordem	<code>math.perm(n,r)</code>

14. CHECKLIST ANTES DE RESPONDER

- O range inclui todos os valores?
- Preciso de um sorteio ou vários?
- Pode haver repetição?
- A ordem dos elementos importa?
- Estou combinando categorias? → pense em `product`.
- Preciso desempacotar argumentos? → pense no `*`.
- Estou usando `values()` para obter as opções?
- Estou chamando a função com `( )`?
- Se há vários experimentos, o sorteio está dentro do laço?
- Distribuição normal → `gauss(media, desvio)`.

15. MINI-COLA FINAL

```
random.uniform(10,45) # real uniforme
random.choice(lista) # 1 item
random.choices(lista, weights=[...], k=1000) # vários, pode repetir
random.sample(range(1,61), k=6) # vários, sem repetir
random.gauss(media, desvio) # normal
list(product(*dicionario.values())) # todas combinações
list(permutations(lista, 3)) # grupos ordenados de 3
math.factorial(5) # 120
math.perm(7,3) # 210
range(1,7) # 1,2,3,4,5,6
```

■ **DICA/ERRO: As 3 perguntas mais importantes:** (1) o limite do range inclui tudo? (2) a ordem importa? (3) o sorteio permite repetição?